Team 6 – M3 Report: Multi-Agent Meeting Action-Item System

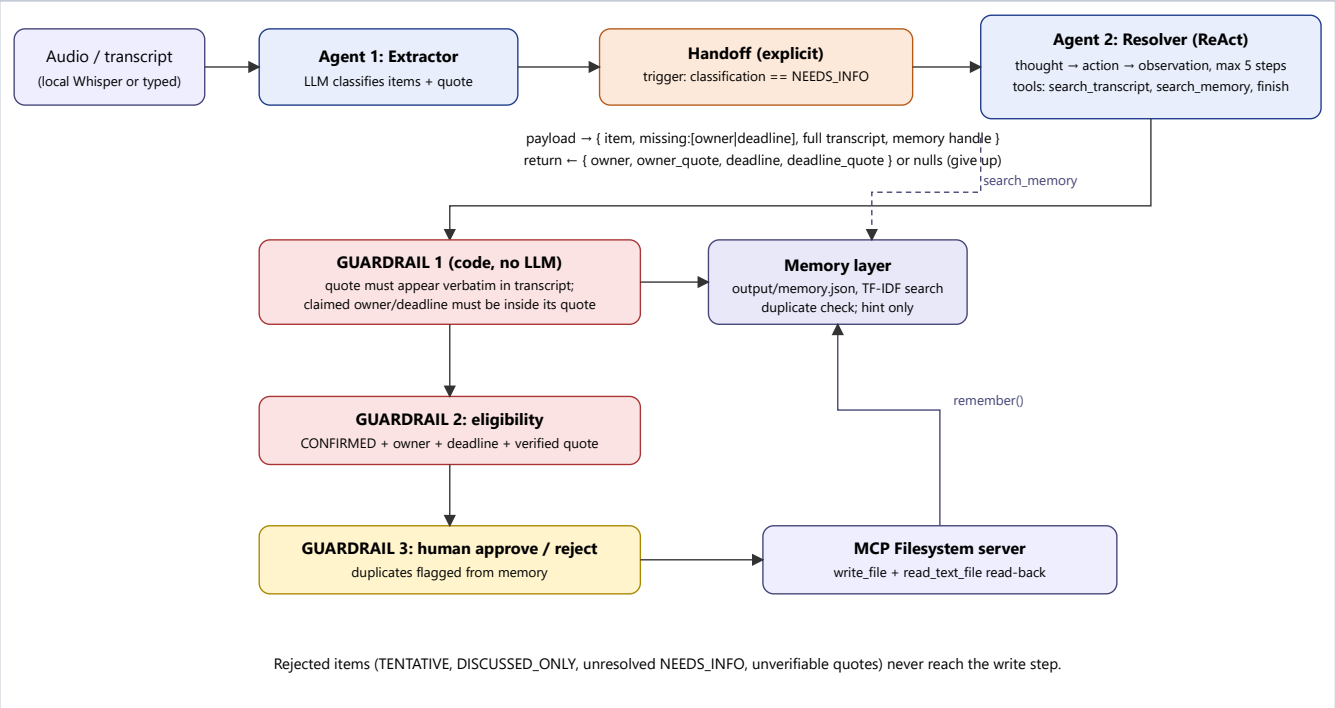
Repository: SamTusick/TeamProject6_AI_Agent | Extractor agent + Resolver agent (ReAct) + persistent memory, with code-level guardrails and human approval

> **Honesty statement.** Cost figures, prices and conversion rates below are **assumptions** we have not yet validated with vendors or users; each is labeled and can be swapped in the formulas. The system is a working prototype, not a polished product. Known failures are listed in §7.

1. What the system does

Small teams lose decisions after meetings because nobody reliably separates a real commitment from a hedge or an idea. Our system reads a meeting transcript (imported, typed, or produced on-device from a recording with the open-source Whisper model in our desktop app) and produces **human-approved action items**, each tied to a verbatim quote. Approved items are written through the official MCP Filesystem server and also stored in a memory layer for later meetings.

2. Updated architecture



Roles, handoff and reasoning

Component	Detail
Extractor agent	`backend/agents/extractor.py`. Single LLM call (NRP-hosted model; `qwen3` / `glm-5`). Classifies candidates as CONFIRMED, TENTATIVE, DISCUSSED_ONLY, NEEDS_INFO; the prompt biases toward the lower-confidence label.
Resolver agent	`backend/agents/resolver.py`. Separate system prompt and loop. Only job: find a missing owner/deadline stated elsewhere in the transcript, or report that it is not stated.

Handoff trigger	`backend/agents/handoff.py`. The Extractor emits `NEEDS_INFO` (decisive language but missing owner and/or deadline).
Handoff payload	Item (title, quote), list of missing fields, full transcript, memory handle. The return value is the same item, upgraded to CONFIRMED only when both fields have verified quotes.
Reasoning pattern	ReAct, implemented explicitly: JSON `{thought, action, input}` per turn; observations fed back; hard cap of 5 steps; unusable replies stop the loop.
Knowledge / memory	`backend/core/memory.py`. Persistent JSON store of human-approved items with lexical TF-IDF retrieval (RAG-lite, no embeddings). Used for `search_memory` hints and cross-meeting duplicate detection.
Guardrails	(1) quote verification and value-in-quote check in code (`backend/core/guardrails.py`), (2) eligibility rule, (3) human confirmation, (4) memory is never evidence, (5) MCP server sandboxed to `./output`, (6) consent prompts before recording or transcribing.

Code organization (backend at the top, frontend at the bottom)

```
backend/            Python: agents/ (extractor, resolver, handoff) · core/ (config, llm_client, guardrails, memory)
                    tools/ (MCP client) · speech/ (Whisper + own model) · workers/ (processes the app launches)
docs/               setup guides and this report
tests/              backend/ unit tests · frontend/ unit + Electron e2e tests · run_tests.py live suite
frontend/main/      Electron main process: controllers/ · services/ · models/ · preload
frontend/renderer/  React UI: components/ · hooks/ · lib/ · css/ (split stylesheets)
```

3. Evaluation plan – 10 test cases

Pass criteria are automatic where possible. Cases T1–T6 are live end-to-end (real model + real MCP server) in `tests/run_tests.py`; T7–T10 are offline/unit or manual. Because the model is non-deterministic, live cases are run 3 times and must pass 3/3 (or we report the failure rate).

#	Test case	Input	Expected result	Measures
T1	Clear commitment	`a_confirmed`	CONFIRMED item with owner+deadline, verified quote, file written and read back	Happy path, MCP write
T2	Hedged commitment	`b_tentative` ("might", "I'll try")	TENTATIVE; nothing written	False-CONFIRMED rate (target 0)
T3	Idea only	`c_discussed_only`	DISCUSSED_ONLY; nothing written	False-CONFIRMED rate
T4	Decision with no owner/deadline anywhere	`d_needs_info`	Handoff fires, Resolver gives up with nulls, item stays NEEDS_INFO, clarifying question shown, nothing written	Handoff trigger; Resolver does not invent
T5	Multiple commitments	`e_multiple_confirmed`	2 CONFIRMED items, different owners/deadlines, both written	Recall on multi-item
T6	Owner/deadline stated later in the meeting	`f_resolvable`	Item reaches CONFIRMED (directly, or via Resolver with quotes) and is written	Resolver success / end-to-end
T7	Resolver hallucination	Scripted fake LLM returns an invented quote	Claim rejected, item stays NEEDS_INFO	Guardrail effectiveness (unit)
T8	Memory is not evidence	Memory says "Jordan"; transcript has no owner	Owner not applied	Memory-poisoning resistance (unit)

T9	Cross-meeting duplicate	Same commitment approved in meeting 1, repeated in meeting 2	Flagged `[memory]`; skipped under `--approve-all`; memory persists across runs	Memory layer value
T10	Speech-to-text quality	Recorded meeting transcribed with local Whisper; LibriSpeech clip as reference	Quotes verify against the produced transcript; word errors measured against a known reference; multi-speaker audio checked manually for lost or merged content	Transcription effect on accuracy

Metrics we will report: precision of items written (target ≥ 95%, because every written item must be a real commitment), recall of true commitments (target ≥ 70% – we value precision over recall), false-CONFIRMED count (target 0), handoff success rate, steps used by the Resolver, seconds per transcript, cost per transcript. Offline tests T7–T9 are in `tests/backend/test_agents.py` (13 tests, passing), and the whole Python suite is 30 passing tests.

4. Operational cost model (100 / 1,000 / 10,000 users)

Assumptions (all replaceable)

- Usage: 8 meetings per user per month, 30 minutes each; transcript ≈ 7,000 input tokens.
- LLM price (assumed commercial mid-tier model, since the NRP endpoint is a free research platform and not a production service): **$0.50 / M input tokens, $2.00 / M output tokens**.
- Extractor: 7,000 in + 800 out ≈ **$0.0051**/meeting. The Resolver fires on ~30% of meetings (assumption): 3 steps × 7,200 in + 150 out ≈ $0.0117 → $0.0035 expected. **LLM ≈ $0.0086/meeting.**
- **Speech-to-text (base case): $0 to us.** The desktop app runs open-source Whisper on the user's own computer, so there is no per-minute API cost. The trade-off is the user's hardware and a one-time ~150 MB model download.
- **Speech-to-text (alternative: hosted Whisper API for a web or mobile version):** assumed **$0.006 per audio minute** for 50% of meetings.
- Hosting/infra ($40 / $250 / $2,200) and other per-user costs (storage, email, logging: $0.10/user) are rough estimates.
- Price assumption for margin only: $12 per user per month.

	100 users	**1,000 users**	**10,000 users**
Meetings / month	800	8,000	80,000
LLM (Extractor + Resolver)	$6.89	$68.88	$688.80
Hosting / database / queue	$40	$250	$2,200
Storage, email, logging ($0.10/user)	$10	$100	$1,000
Speech-to-text, local Whisper (base case)	$0	$0	$0
Total monthly cost (base case)	**$56.89**	**$418.88**	**$3,888.80**
Cost per user / month	$0.57	$0.42	$0.39
Revenue if all pay $12	$1,200	$12,000	$120,000
Gross margin (before payroll, support, payment fees)	95%	97%	97%
Alternative: + hosted Whisper API (50% of meetings)	+$72.00	+$720.00	+$7,200.00
Total with hosted Whisper	**$128.89**	**$1,138.88**	**$11,088.80**
Cost per user / margin with hosted Whisper	$1.29 / 89%	$1.14 / 91%	$1.11 / 91%

What this tells us. Running speech-to-text on the user's device removes the largest variable cost; the LLM is then under 20% of cost at every scale and fixed hosting dominates. If the Resolver fires on 100% of meetings instead of 30%, LLM cost rises about 2.4× (≈ $0.02/meeting) – still small. **Sensitivity:** if only 25% of 100 users pay, revenue is

$300 against $57 cost – positive, but payroll is not covered; at this scale the product is not self-sustaining without outside funding. Risks not modeled: model price changes, retries after failures, abuse, a free tier, and support cost for users whose computers are too slow for local Whisper.

5. Path to sustainability

Stage	Approach	Evidence needed before moving on
0–100 users (pilot)	Free for pilot teams; founders absorb ~$60/month; on-device Whisper keeps costs minimal.	≥ 30 teams use it weekly; precision ≥ 95% on their meetings.
100–1,000	Paid plan (assumed $8–15 per user/month) for small teams; free tier limited to 3 meetings/month.	≥ 10% free→paid conversion; cost per user under $2.
1,000–10,000	Team plan with integrations (Jira / Google Calendar – planned, not built); annual billing; negotiate LLM volume pricing; move to a dedicated database for memory; optional hosted transcription for users without capable hardware.	Gross margin ≥ 70% after support; monthly churn under 5%.

Sustainability risks and mitigations. (1) Vendor dependence on LLM pricing – keep the model name in `.env` so models are swappable. (2) Privacy – recordings of people on calls: consent prompts, local storage and local transcription by default, no audio leaves the device, documented retention. (3) Accuracy trust – one false "decision" damages credibility, so we favor precision and always show the source quote. (4) Environmental footprint – small models, short prompts, a capped 5-step Resolver, and a small on-device Whisper model keep inference light. (5) Team sustainability – clear ownership by folder (backend, frontend, business docs) and automated tests keep handoffs between teammates safe.

6. User testing plan

- **Participants:** 8–10 people from 3–4 small teams (student project groups and one small business), each running at least 2 real meetings. Consent for recording is obtained from every attendee; participants can delete data at any time.
- **Protocol:** Week 1 – 30-minute onboarding; participants record or import a meeting. Week 2 – they review the extracted items, approve/reject, and record whether each item was correct. We observe one session per team (think-aloud) without coaching.
- **Measures:** precision and recall against participants' own list of real commitments (collected independently before they see the output); time from meeting end to approved list vs. their usual method; number of rejected/edited items; SUS usability score (target ≥ 68); trust question "Would you act on this item without checking the quote?"; willingness to pay (ask for a price, not a yes/no); Whisper word-error observations on their real audio.
- **Success criteria:** ≥ 90% of approved items judged correct; ≥ 60% of true commitments found; ≥ 6 of 10 participants say they would use it again; no unconsented recording incidents.
- **Ethics / data handling:** store transcripts locally; anonymize names in anything shared in the report; delete the data after analysis.
- **What we will do with failures:** every false item or missed commitment is logged with the transcript snippet and added as a regression test.

7. Limitations and failures we have observed (honest section)

- **Model endpoint outage.** During testing the default `qwen3` endpoint hung for 5+ minutes on even a trivial prompt, while `glm-5` answered in ~3 s. We switched models via `MODEL_NAME`; the agent has no timeout/fallback yet – an open bug.
- **The handoff is rare on easy transcripts.** On `f_resolvable` the Extractor resolved owner and deadline itself, so no handoff occurred. We verified the Resolver by running it directly on that transcript (it found both with quotes) and on `d_needs_info` (it correctly gave up). Different models also reason differently: `glm-5` searched the transcript before answering, while `qwen3` answered after one thought.
- **Resolver quotes can span a speaker label** (e.g. "Morgan: Thanks. Back to the pricing page …"); the check accepts this because it is verbatim text.
- **Memory is lexical, and the desktop app does not write to it.** A paraphrased duplicate is missed; only items approved through the terminal CLI are remembered (the desktop app reads memory but does not save approvals to it); no concurrency control.
- **Speech-to-text is moderately accurate.** Whisper base transcribed a LibriSpeech test clip with a few word errors ("were sleeping" became "are sleeping"; "a cooing dove" became "accuing dove"). Real multi-speaker calls have not been tested. The first run needs internet to download the model. We first built a Wispr Flow cloud integration but removed it because Wispr's API is closed to new users, so it could never be tested against the live service.
- **Non-determinism.** The same transcript can give different labels on different runs.
- **Scale.** Single-user JSON memory; no auth or multi-user backend yet, so the 1,000 and 10,000-user rows are cost estimates, not tested capacity.

8. Team contributions (from the Git history)

Member	Commits (main + tony-update)	Work
Samuel Tusick	14	Repository setup, MCP filesystem integration, verification guardrail, test runner, Whisper integration, dependency pinning, pull-request merges
Ishita Chakkalakkal	5	Multi-item test case, chained memory-then-transcript resolver test, README test-count updates
Tony Nguyen	2 + this restructure	Resolver agent, ReAct loop, memory layer, handoff, desktop app, code reorganization, this report

The assignment requires visible commits from both CS and entrepreneurship members. The cost model, sustainability path and user testing plan in this report are the entrepreneurship deliverables; the team should make sure those members commit their edits (for example to `docs/report/` or the README CHANGES section) so the history shows both tracks.